

Rapport d'alternance

Titre : Concepteur Développeur d'Applications

Nom : Le Floch Ewen

Entreprise : OPEN

Période : Septembre 2024 – Août 2026

Projet présenté : One Piece Zero



Table des matières

TABLE DES MATIERES.....	2
REMERCIEMENTS.....	4
QUI SUIS-JE ?	5
PRESENTATION DE L'ENTREPRISE	6
CONTEXTE DE L'ALTERNANCE.....	7
FICHE DE COMPETENCES	8
CAHIER DES CHARGES.....	9
1. PRESENTATION DU PROJET	9
2. OBJECTIFS DU PROJET	9
3. PERIMETRE DU PROJET.....	10
4. IDENTIFICATION DES ACTEURS.....	12
5. FONCTIONNALITES DETAILLEES.....	12
6. REGLES METIER	14
7. ARCHITECTURE TECHNIQUE	15
8. MODELISATION DES DONNEES	16
9. SECURITE.....	17
10. TESTS.....	17
11. ÉVOLUTIONS ENVISAGEES	18
GESTION DE PROJET	18
1. ORGANISATION DU PROJET	18
2. PLANNING ET SUIVI	19
3. ENVIRONNEMENT HUMAIN ET TECHNIQUE	21
4. OBJECTIFS QUALITE.....	22
SPECIFICATIONS FONCTIONNELLES	24
1. IDENTIFICATION DES ACTEURS.....	24
2. CAS D'UTILISATION PRINCIPAUX	24
3. USER STORIES.....	26

4. DIAGRAMME DE CAS D'UTILISATION UML.....	27
5. SCENARIO FONCTIONNEL PRINCIPAL : COMBAT CONTRE UN ENNEMI.....	28
6. REGLES METIER	29
SPECIFICATIONS TECHNIQUES	30
1. ARCHITECTURE GENERALE DU PROJET	30
2. TECHNOLOGIES UTILISEES	30
3. ARCHITECTURE APPLICATIVE	31
4. COMMUNICATION API REST	32
5. GESTION DES DONNEES	33
6. SECURITE TECHNIQUE.....	33
7. ORGANISATION DU CODE SOURCE.....	34
8. SCHEMA D'ARCHITECTURE.....	34
REALISATIONS DU CANDIDAT	37
1. CONCEPTION DE L'APPLICATION	37
2. DEVELOPPEMENT DU BACK-END	41
3. SECURISATION DE L'APPLICATION	48
4. DEVELOPPEMENT DU FRONT-END.....	50
5. DEPLOIEMENT ET CONFIGURATION DE L'APPLICATION	53
6. TESTS ET VALIDATION DE L'APPLICATION.....	54
7. CONCLUSION ET PERSPECTIVES.....	58

Remerciements

Je tiens tout d'abord à remercier Olivia et Sarah, qui m'ont sélectionné pour participer au Hackathon 2024 organisé par France Travail et m'ont proposé de poursuivre l'aventure au sein de leur entreprise.

Je souhaite également remercier l'ensemble du pôle Drupal, et particulièrement Loïc, François et Nathalie qui m'ont beaucoup appris durant ces deux années et m'ont très souvent guidé. Je remercie également mon manager Olivier pour son accompagnement tout au long de l'alternance.

Je remercie aussi l'ensemble des équipes des différents projets sur lesquels j'ai à chaque fois eu la chance d'être bien accueilli et accompagné.

Enfin, je tiens à saluer les équipes pédagogique et administrative de l'ENI, les formateurs externes ainsi que mes collègues de formation pour ces 2 années riches en apprentissage, en bienveillance et en bons moments partagés.

Qui suis-je ?

Depuis mes années de collège, où je découvrais mes premiers jeux par navigateur et les tutoriels du *Site du Zéro*, j'ai développé un intérêt durable pour la programmation web. Bien que mon orientation après le lycée ait été marquée par plusieurs essais dans différentes filières, je revenais régulièrement vers le développement, preuve que ce domaine représentait pour moi bien plus qu'un simple loisir.

En 2019, ma réussite au concours d'entrée de l'École 42 a renforcé ma conviction de vouloir évoluer dans le numérique. Ne pouvant toutefois intégrer l'école immédiatement, j'ai travaillé durant trois ans comme assistant d'éducation afin de financer une première formation diplômante : le titre professionnel de niveau 5 *Développeur Intégrateur Web* au sein d'OpenClassrooms. Cette formation m'a permis d'acquérir des bases solides, tout en confirmant mon envie d'approfondir mes compétences.

Après l'obtention de mon diplôme, les échanges avec plusieurs recruteurs m'ont encouragé à poursuivre mes études afin d'élargir mon profil. En janvier 2024, ma participation à un hackathon organisé par France Travail a marqué un tournant dans mon parcours : l'entreprise Open, rencontrée à cette occasion, m'a proposé une alternance à la suite de mon inscription à l'ENI.

J'ai ainsi intégré, en septembre 2024, la formation *Concepteur Développeur d'Applications* afin de monter en compétence sur l'architecture logicielle, le design applicatif et les technologies backend. Cette alternance représente une étape clé dans mon projet professionnel : devenir un développeur capable de concevoir, structurer et maintenir des applications complètes, tout en évoluant dans un environnement stimulant et exigeant.

Présentation de l'entreprise

Open, un industriel de l'innovation au service de votre compétitivité

Open est une entreprise française créée en **1989**. L'entreprise, telle qu'on la connaît aujourd'hui est née du rapprochement de plusieurs sociétés de services informatiques telles que Teamlog, Innetis et Syllis.

Quelques chiffres clés permettent d'illustrer l'importance du groupe :

- **4000** collaborateurs engagés, experts et enthousiastes
- **420** millions d'euros de chiffre d'affaires
- **16** implantations dont **14** en France
- Plus de **500** clients actifs, parmi lesquels plus de **+50%** des entreprises du CAC40
- Un positionnement dans le Top **20** des ESN françaises

Présent depuis plus de **20** ans à Rennes, Open s'impose aujourd'hui comme un acteur reconnu de la transformation digitale au niveau local. Le centre de production Rennes-Lannion regroupe plus de **300** collaborateurs et s'appuie notamment sur des expertises **Cloud** et **DevOps**, ainsi que sur une expertise **omni-canal** couvrant l'ergonomie, le design et le développement d'applications web et mobiles (Android et iPhone), destinées à différents supports tels que les smartphones, les tablettes ou encore les services vocaux interactifs.

Contexte de l'alternance

Dans le cadre de mon alternance de deux ans en tant que Concepteur Développeur d'Applications chez Open, j'ai d'abord intégré le pôle Drupal. En raison d'un contexte économique particulier, ma première mission a consisté à travailler en autonomie avec l'accompagnement ponctuel d'un architecte applicatif, d'un responsable technique et d'un responsable CI/CD, sur la création d'un outil PHP de génération de rapports de sécurité intégré à un pipeline GitLab. Cet outil permet de produire, de manière régulière ou manuelle, des rapports destinés aux clients.

Par la suite, après avoir commencé un second projet Python portant sur l'analyse des interdépendances entre modules et bibliothèques de différents projets et langages, j'ai intégré un projet Angular / Symfony consacré à la refonte et à la montée de version de *Sites & Organismes* de Rennes Métropole.

Dans ce cadre, mon rôle en tant que développeur front-end consistait principalement à intégrer les différents blocs du site à partir des maquettes fournies. J'ai également été amené à accompagner ponctuellement le développeur back-end dans la correction des anomalies rencontrées lors de la montée de version de l'API.

Enfin, depuis plusieurs mois, j'interviens en tant que développeur fullstack sur le projet anonymisé d'une Administration Nationale Publique, au sein d'une équipe d'une vingtaine de personnes. Cette expérience m'a permis de travailler sur des scripts SQL, du développement back-end en Java ainsi que du développement front-end en Angular. Elle m'a également apporté une meilleure compréhension de l'architecture de projet d'envergure, celui-ci reposant sur plusieurs applications interconnectées, composées de multiples microservices et dépendances avec d'autres projets développés par différents prestataires.

Bien que cette alternance m'ait permis de mettre en œuvre une grande partie des compétences attendues dans la formation, j'ai choisi de présenter dans ce dossier un projet personnel regroupant l'ensemble de ces compétences.

Fiche de compétences

Nom et prénom du candidat : Ewen LE FLOCH

Document complété d'un commun accord entre le stagiaire et le responsable du stage en entreprise à joindre au rapport d'activité.

Compétences Voir le détail dans le référentiel d'emploi, d'activités et de compétences	Cocher les compétences mises en œuvre lors du projet en entreprise (en totalité ou partiellement)
C1 - Installer et configurer son environnement de travail en fonction du projet	☒
C2 - Développer des interfaces utilisateur	☒
C3 - Développer des composants métier	☒
C4 - Contribuer à la gestion d'un projet informatique	☒
C5 - Analyser les besoins et maquetter une application	☒
C6 - Définir l'architecture logicielle d'une application	☐
C7 - Concevoir et mettre en place une base de données relationnelle	☐
C8 - Développer des composants d'accès aux données SQL et NoSQL	☒
C9 - Préparer et exécuter les plans de tests d'une application	☒
C10 - Préparer et documenter le déploiement d'une application	☒
C11 - Contribuer à la mise en production dans une démarche DevOps	☒

Observations éventuelles :

Du stagiaire :
.....
.....
.....

De l'entreprise :
Ewen a participé à plusieurs projets: internes pour la mise en oeuvre d'outils, et projets client pour des développements angular, front et back.
Il s'est très bien intégré à l'équipe, a fourni un reporting à la cible de ce qui est demandé. De plus, il est également volontaire pour la prise des sujets.
.....

Nom et Prénom du stagiaire :

LE FLOCH Ewen

Signature :



Entreprise : Open

Nom du responsable de stage :

Olivier Gougeon

Signature :



Cahier des charges

1. Présentation du projet

Le projet consiste à concevoir et développer une application web de type jeu de rôle accessible via navigateur, inspirée de l'univers du manga One Piece.

Ce projet est réalisé dans le cadre de la validation du titre Concepteur Développeur d'Applications. Il a pour objectif de mettre en œuvre l'ensemble des compétences attendues dans le cadre du référentiel CDA, notamment :

- L'analyse des besoins
- La conception d'une architecture logicielle
- Le développement d'une application web complète
- La mise en place d'une base de données relationnelle
- La sécurisation des échanges et des données
- La gestion d'un projet informatique

Le choix d'un RPG par navigateur permet d'intégrer plusieurs problématiques techniques et métier au sein d'un même projet :

- La gestion des utilisateurs
- La persistance des données
- Une logique métier complexe
- Un système de progression
- Une gestion des combats
- Une communication entre un front-end et une API REST

Le projet a volontairement été limité à une première version fonctionnelle (MVP – *Minimum Viable Product*) afin de se concentrer sur les fonctionnalités essentielles et de garantir la stabilité globale de l'application.

2. Objectifs du projet

Les principaux objectifs du projet sont les suivants :

- Développer une application web complète reposant sur une architecture front-end / back-end
- Concevoir une API REST sécurisée

- Implémenter une logique métier complexe autour du système de combat et de progression
- Concevoir une base de données relationnelle cohérente et évolutive
- Assurer la persistance des données utilisateur
- Structurer le projet selon une architecture maintenable et évolutive
- Mettre en œuvre des mécanismes de sécurité adaptés à une application web moderne

3. Périmètre du projet

Fonctionnalités incluses dans le MVP :

La première version du projet comprend les fonctionnalités suivantes :

Gestion des utilisateurs

- L'inscription d'un utilisateur
- Une connexion sécurisée
- Une création automatique d'une progression liée au compte avec un personnage de départ

Interface publique

- La consultation des actualités du jeu
- La présentation de l'univers et du projet

Monde du jeu

- L'accès à une première île jouable : Île de Dawn
- La présence d'une première zone : le Village de Fuchsia

Système de combat

- Des combats en solo au tour par tour
- La gestion des points de vie
- La gestion de l'endurance
- L'utilisation de capacités

Progression

- Le gain d'expérience
- La montée de niveau
- Le déblocage progressif du contenu

Boss de fin de MVP

- Un combat contre le boss Higuma accessible au niveau 6 débloquent la zone / île suivante

Système de factions

Après le combat contre Higuma, le joueur peut choisir une faction :

- Pirate
- Marine
- Révolutionnaire
- Chasseur de primes

Ce choix est définitif dans la version actuelle du projet.

Fonctionnalités hors périmètre

Certaines fonctionnalités ont volontairement été exclues du MVP afin de limiter la complexité du projet et de prioriser les mécaniques principales :

- Le mode multijoueur
- Le système d'équipages
- Le chat instantané et le forum
- Un système économique avancé
- Les commerces entre joueurs
- Un système de quêtes avancé

Ces fonctionnalités pourront être intégrées dans de futures évolutions du projet.

4. Identification des acteurs

L'application distingue trois types d'acteurs principaux.

Acteur	Description
Visiteur	Utilisateur non authentifié pouvant consulter les actualités et créer un compte
Joueur	Utilisateur authentifié pouvant accéder au jeu et progresser
Administrateur	Utilisateur disposant de droits supplémentaires permettant la gestion du contenu

5. Fonctionnalités détaillées

Gestion des comptes

Le système de comptes permet :

- La création sécurisée d'un compte utilisateur
- L'authentification via connexion
- La persistance des informations utilisateur en base de données

Chaque utilisateur possède :

- Un identifiant unique
- Un pseudo
- Une adresse e-mail
- Un mot de passe haché

Gestion du personnage

Chaque joueur possède une progression associée à un personnage jouable sélectionné, disposant de plusieurs statistiques :

- Son niveau
- Son expérience
- Sa puissance
- Son endurance

- Sa prime
- Son argent (berries)
- Sa faction

Ces statistiques évoluent en fonction de la progression du joueur.

Monde du jeu

Le monde est structuré en :

- Îles
- Zones
- Ennemis accessibles selon le niveau du joueur

La première version du projet comprend :

- L'île de Dawn
- Le village de Fuchsia

Système de combat

Le système de combat constitue la fonctionnalité centrale de l'application. Il représente également la fonctionnalité métier la plus complexe du projet. Il nécessite la gestion des statistiques du joueur, des effets temporaires, de l'endurance, des calculs de dégâts ainsi que des interactions entre les différentes capacités disponibles.

Le joueur peut utiliser plusieurs capacités différentes :

- L'attaque
- Le soin
- Le boost (augmentation des dégâts)
- L'esquive
- Le contre (esquive l'attaque ennemie suivante et attaque l'adversaire)
- Le renvoi (retourne l'attaque de l'adversaire contre lui)
- La paralysie

Chaque capacité possède :

- Un coût en endurance
- Des effets spécifiques
- Des impacts différents sur le déroulement du combat

Le système applique ensuite les calculs nécessaires afin de déterminer :

- Les dégâts infligés
- Les effets appliqués
- Le résultat du tour du combat

Progression

Le système de progression permet :

- De gagner de l'expérience après un combat
- D'augmenter le niveau du joueur
- D'augmenter la prime du joueur
- De débloquer progressivement le contenu du jeu

Le niveau du joueur influence notamment :

- L'accès aux îles
- L'accès aux zones
- L'accès au boss
- Les statistiques du personnage

Boss de fin de MVP

Le boss Higuma devient accessible à partir du niveau 6. Ce combat représente la validation de la première phase de progression du joueur et constitue l'objectif principal du MVP.

Après sa victoire, le joueur débloquent l'accès à la zone suivante ainsi qu'au système de factions.

Système de factions

Après le combat contre Higuma, le joueur peut choisir une faction :

- Pirate
- Marine
- Révolutionnaire
- Chasseur de primes

Ce choix est définitif dans la version actuelle du projet. Il permet de renforcer l'impact des décisions du joueur sur sa progression et son identité dans le jeu.

6. Règles métier

Les principales règles métier du projet sont les suivantes :

Règle métier	Description
Accès aux zones	Certaines zones nécessitent un

	niveau minimum
Combat	Un combat nécessite une endurance suffisante
Capacités	Chaque capacité consomme de l'endurance
Progression	Une victoire permet de gagner de l'expérience
Boss Higuma	Accessible uniquement à partir du niveau 6. La victoire permet de débloquent le système de factions
Faction	Le choix de faction est définitif
Persistance	Les données du joueur et de sa progression sont sauvegardées en base de données

Ces règles permettent de structurer la logique métier de l'application et de garantir la cohérence du système de progression.

7. Architecture technique

L'application repose sur une architecture en couches séparant les différentes responsabilités du système.

Front-end

Le front-end est responsable de l'affichage des interfaces utilisateur, des interactions avec le joueur ainsi que de la communication avec l'API REST.

Il est développé avec **Angular** et **TypeScript**. Cette architecture permet de structurer l'interface sous forme de composants réutilisables et de faciliter la séparation entre l'affichage, les traitements et les appels à l'API.

Back-end

Le back-end, développé avec **Java** et **Spring Boot**, centralise l'ensemble de la logique métier de l'application.

Il gère notamment :

- Les combats
- La progression du joueur
- Les traitements de sécurité
- Les échanges avec la base de données

L'application repose sur une architecture REST permettant la communication entre le front-end et le back-end via des échanges JSON. Cette séparation permet une meilleure maintenabilité, une meilleure évolutivité et une réduction du couplage entre les différentes parties de l'application.

Base de données

La persistance des données repose sur une base relationnelle **MySQL** permettant le stockage :

- Des utilisateurs
- Des personnages
- Des combats
- Des îles
- Des zones
- Des ennemis
- Des capacités

L'organisation des données a été pensée afin de garantir la cohérence des informations liées à la progression du joueur et au système de combat.

8. Modélisation des données

L'application s'appuie sur plusieurs entités principales :

- Utilisateur
- Personnage
- Progression du joueur
- Ennemi
- Combat
- Zone
- Île
- Capacités
- Factions
- Actualités

La modélisation a été réalisée à partir des besoins du métier, des mécaniques de progression ainsi que du système de combat. Elle a permis de concevoir une structure relationnelle cohérente et évolutive.

9. Sécurité

Plusieurs mécanismes de sécurité ont été intégrés au projet :

- Hachage des mots de passe avec BCrypt
- Validation des données utilisateur
- Authentification par JWT
- Gestion des rôles utilisateurs
- Protection des routes de l'API via Spring Security
- Contrôle des accès aux ressources protégées
- Protection contre les données invalides

Une attention particulière a également été portée à la cohérence des données métier afin d'éviter les comportements incohérents ou les manipulations invalides.

10. Tests

Différents types de tests ont été réalisés afin de garantir le bon fonctionnement de l'application :

Tests unitaires

Ils permettent de vérifier les calculs des dégâts, les règles de progression et la gestion de l'endurance.

Tests d'intégration

Ils permettent de vérifier les échanges entre l'API et la base de données ainsi que le fonctionnement global des échanges entre les couches de l'application.

Tests fonctionnels

Ils permettent de tester le déroulement des combats, la progression du joueur, l'accès aux zones et au boss ainsi que le système de faction.

Des jeux d'essais spécifiques ont notamment été réalisés pour le combat contre le boss.

11. Évolutions envisagées

Plusieurs évolutions sont envisagées pour les futures versions du projet :

- Le mode multijoueur
- La création et l'intégration d'équipages
- De nouvelles îles
- De nouvelles zones
- De nouveaux ennemis et boss
- Un système économique
- Un système de quêtes
- Les interactions entre joueurs

A partir de ce cahier des charges, le projet a été organisé selon une méthode de travail permettant de planifier, suivre et réaliser les différentes fonctionnalités de l'application.

Gestion de projet

1. Organisation du projet

Pour le développement de ce projet, j'ai choisi d'utiliser une méthode Agile de type Scrum. Cette approche, basée sur un développement itératif et progressif, m'a permis d'organiser les différentes étapes du projet tout en conservant une certaine flexibilité dans l'évolution des fonctionnalités.

Le projet a été structuré autour d'un backlog regroupant les principales fonctionnalités à développer sous forme de *user stories*. Ces fonctionnalités ont ensuite été découpées en tâches techniques afin de faciliter leur planification, leur suivi et leur réalisation.

Le développement a notamment été organisé autour des fonctionnalités suivantes :

- La gestion des utilisateurs ;
- La gestion des personnages ;
- Le système de combat ;
- La gestion des statistiques du joueur ;

- La persistance des données.

Le backlog a évolué tout au long du développement afin d'intégrer les ajustements réalisés sur le système de combat, la progression du joueur ainsi que sur la modélisation des données.

Le suivi du projet a été réalisé à l'aide d'un tableau de tâches organisé en plusieurs colonnes :

- To Do
- In Progress
- Done

Cette organisation m'a permis de visualiser l'état d'avancement du projet, d'identifier rapidement les blocages et de réorganiser certaines priorités lorsque cela était nécessaire.

Dans le cadre de ce projet individuel, plusieurs éléments de Scrum ont été adaptés afin de correspondre à une organisation en autonomie. Un suivi écrit régulier m'a notamment permis de consigner les fonctionnalités développées, les difficultés rencontrées ainsi que les ajustements réalisés au cours du projet.

Cette méthode de travail s'est révélée particulièrement adaptée au caractère évolutif du projet, notamment pour le système de combat et la modélisation des données, qui ont nécessité plusieurs modifications au fur et à mesure du développement.

2. Planning et suivi

Le développement du projet a été organisé en plusieurs sprints courts afin de permettre une progression itérative et une validation régulière des fonctionnalités développées.

Les premières phases du projet ont principalement été consacrées à la conception de la base de données, à la réalisation du MCD et du MPD, à la mise en place de l'architecture du projet et à la définition des principales mécaniques métier.

Les fonctionnalités ont ensuite été développées progressivement selon leur priorité et leur importance dans le fonctionnement global de l'application.

Les principales étapes du développement ont été les suivantes :

- 1. La conception de la base de données et réalisation du MCD / MPD**
- 2. La mise en place de l'architecture du projet**
- 3. Le développement du système de combat**

4. L'implémentation des différentes capacités en combat
5. La gestion de la progression du joueur
6. La réalisation des tests et correction des anomalies

Le suivi des tâches a été réalisé à l'aide d'un tableau Jira organisé en différentes colonnes (*To Do*, *In Progress* et *Done*). Cet outil m'a permis de suivre l'avancement du projet, d'identifier rapidement les blocages et de réorganiser certaines priorités lorsque cela était nécessaire.

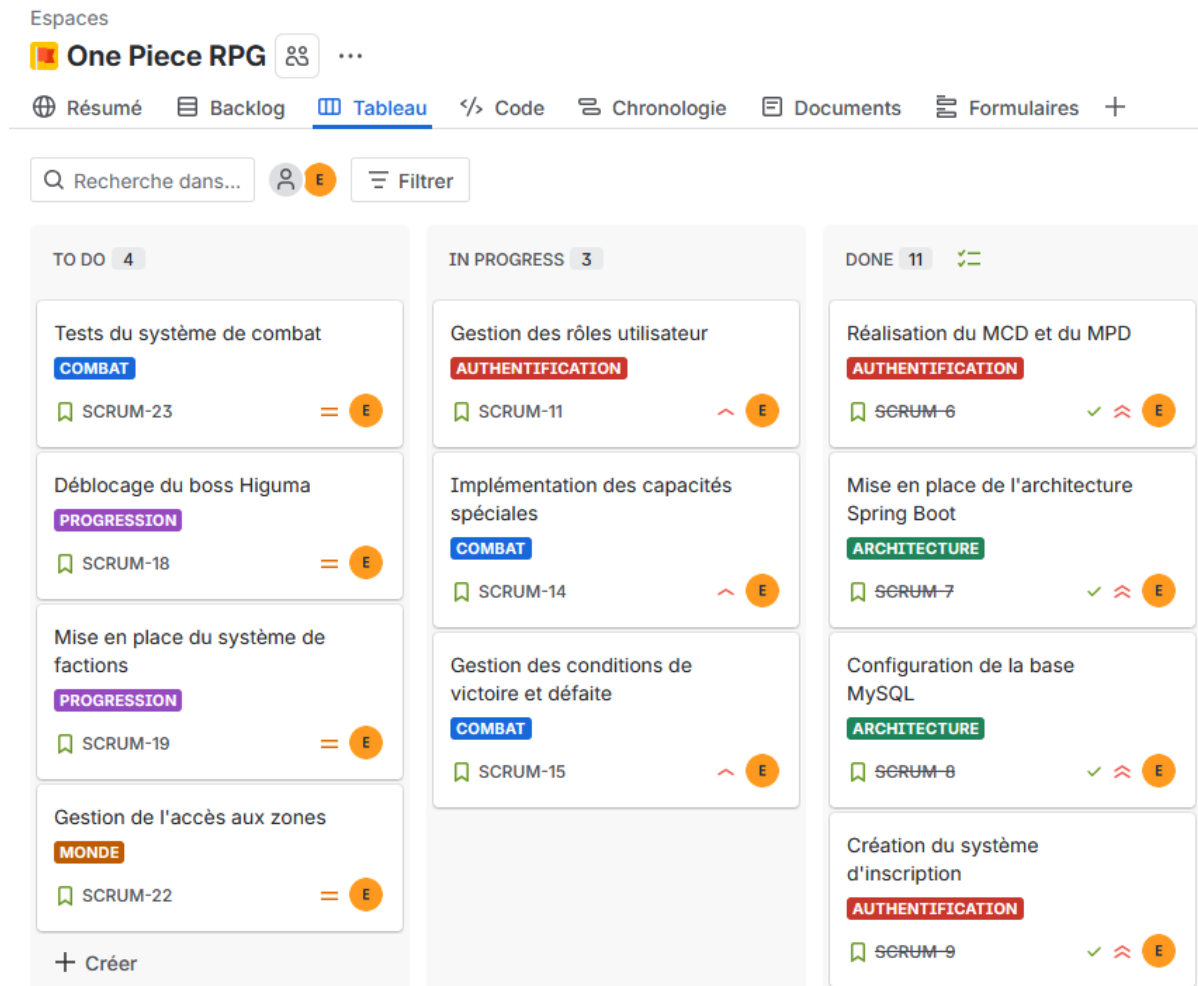


Figure 1 – Tableau Jira utilisé pour le suivi et l'organisation des tâches du projet One Piece Zero

Au cours du développement, plusieurs ajustements ont été réalisés afin d'améliorer certaines mécaniques métier et de faire évoluer le modèle de données. Le système de combat a notamment nécessité plusieurs modifications afin d'équilibrer les différentes capacités disponibles et de mieux gérer les interactions entre les statistiques du joueur et les effets des capacités.

La structure de certaines tables a également évolué au cours du développement afin de mieux représenter les différentes mécaniques métier et de faciliter l'évolution future du projet.

Cette organisation a permis de conserver une bonne maîtrise du développement tout en facilitant l'adaptation aux contraintes rencontrées durant le projet.

3. Environnement humain et technique

Environnement humain

Le projet a été réalisé en autonomie, dans un rôle de concepteur développeur d'applications full-stack. Cette organisation m'a amené à intervenir sur l'ensemble des aspects du projet :

- L'analyse des besoins
- La conception
- La modélisation
- Le développement
- Les tests
- La gestion du projet

Cependant, des échanges réguliers avec les formateurs, l'utilisation des ressources pédagogiques ainsi que la consultation de la documentation technique m'ont permis de valider certains choix techniques et de résoudre les problématiques rencontrées au cours du développement.

Environnement technique

Plusieurs outils et technologies ont été utilisés afin de mener à bien le projet.

Développement

Technologie / outil	Utilisation
Java	Développement back-end
Spring Boot	Développement de l'API REST
TypeScript	Développement front-end
Angular	Interface utilisateur
Visual Studio Code	Environnement de développement

MySQL	Base de données relationnelle
--------------	-------------------------------

Gestion de version	
Outil	Utilisation
Git	Gestion de version
GitHub	Hébergement du dépôt

Modélisation	
Outil	Utilisation
app.diagrams.net	Réalisation du MCD
drawdb.app	Réalisation du MPD
PlantUML	Diagramme de cas d'utilisation
	UML

Gestion de projet	
Outil	Utilisation
Jira	Suivi des tâches et organisation du projet

L'ensemble de ces outils a permis de structurer efficacement le développement du projet, de faciliter le suivi des évolutions et d'assurer une meilleure organisation du travail.

4. Objectifs qualité

Afin de garantir la qualité et la fiabilité du projet, plusieurs bonnes pratiques ont été mises en œuvre tout au long du développement du projet.

Qualité du code

Le projet a été structuré de manière à séparer les différentes responsabilités de l'application entre les couches de présentation, de logique métier et d'accès aux données. Cette organisation a permis de rendre le code plus lisible, de faciliter la maintenance et de simplifier l'évolution future du projet.

Une attention particulière a également été portée au nommage des classes, méthodes et variables, à la cohérence de l'architecture et à la réutilisation des composants.

L'architecture du projet a été pensée afin de faciliter la maintenabilité et l'évolution future de l'application grâce à une séparation claire des responsabilités entre les différentes couches.

Tests

Des tests ont été réalisés tout au long du développement afin de valider le bon fonctionnement des fonctionnalités principales de l'application.

Des tests unitaires ont notamment permis de vérifier les calculs de dégâts, les vérifications d'endurance, les règles de progression ainsi que certaines capacités du système de combat.

Des tests d'intégration ont également été effectués afin de valider les échanges entre l'API REST et la base de données, le fonctionnement global des services et la cohérence des traitements métier.

Des jeux d'essais spécifiques ont également été réalisés afin de tester le combat contre le boss, les conditions d'accès aux zones, le système de faction et différents cas d'erreurs liés aux combats.

Sécurité

Plusieurs mécanismes de contrôle ont été mis en place afin de sécuriser les traitements réalisés par l'application.

Les données reçues par l'API sont vérifiées avant traitement afin d'éviter les valeurs invalides, de limiter les incohérences métier et de prévenir certains comportements inattendus. Une attention particulière a également été portée à la gestion des données utilisateur, à la cohérence des informations enregistrées en base de données et au contrôle des accès des fonctionnalités.

Les mots de passe sont stockés sous forme hachée à l'aide de BCrypt et les routes sensibles de l'API sont protégées afin de limiter les accès non autorisés.

Robustesse

Le développement du projet a nécessité la prise en compte de plusieurs cas particuliers afin d'éviter les erreurs d'exécution et les incohérences métier.

Des contrôles ont notamment été ajoutés afin d'empêcher l'utilisation de capacités lorsque le joueur ne possède pas suffisamment d'endurance, d'éviter des valeurs invalides concernant les points de vie du personnage ou encore d'empêcher certains comportements incohérents durant les combats.

La prise en compte de ces différents cas a permis d'améliorer la stabilité de l'application, la cohérence des mécaniques métier et la maintenabilité globale du projet.

Spécifications fonctionnelles

1. Identification des acteurs

L'application One Piece Zero comporte trois acteurs principaux :

- Visiteur

Le visiteur est un utilisateur non authentifié. Il peut consulter la page d'accueil du jeu, lire les actualités et créer un compte afin de devenir joueur.

- Joueur

Le joueur est un utilisateur authentifié. Il peut accéder à son espace de jeu, consulter sa progression, se déplacer dans le monde disponible, combattre des ennemis, gagner de l'expérience, monter en niveau et choisir une faction.

- Administrateur

L'administrateur dispose de droits supplémentaires. Il peut gérer certains contenus de l'application, notamment les actualités et les éléments liés au jeu.

Ces acteurs correspondent au périmètre défini dans le cahier des charges du projet.

2. Cas d'utilisation principaux

Visiteur

Cas d'utilisation	Description
Consulter les actualités	Le visiteur accède à la page d'accueil et consulte les actualités du jeu.
Créer un compte	Le visiteur renseigne ses informations afin de créer un compte utilisateur.
Se connecter	Le visiteur utilise ses identifiants pour accéder à l'espace joueur.

Joueur

Cas d'utilisation	Description
Accéder au monde	Le joueur accède à l'île et à la zone disponibles selon sa progression.
Consulter sa progression	Le joueur visualise son niveau, son expérience, son endurance, sa prime et son argent.
Combattre un ennemi	Le joueur lance un combat contre un ennemi présent dans la zone.
Utiliser une capacité	Le joueur utilise une capacité durant un affrontement.
Choisir une faction	Le joueur choisit une faction après avoir vaincu le boss Higuma. Ce choix est définitif dans la version actuelle du projet.

Administrateur

Cas d'utilisation	Description
Gérer les actualités	L'administrateur peut créer, modifier ou supprimer les actualités affichées sur la page d'accueil.
Gérer le contenu du jeu	L'administrateur peut intervenir sur les données liées au jeu selon les droits prévus.

3. User stories

Les principales fonctionnalités du projet ont également été formalisées sous forme de user stories afin de représenter les besoins des différents acteurs de l'application.

Acteur	User story
Visiteur	En tant que visiteur, je veux consulter les actualités afin de découvrir les informations du jeu.
Visiteur	En tant que visiteur, je veux créer un compte afin de commencer une partie.
Visiteur	En tant que visiteur, je veux me connecter afin d'accéder à mon espace joueur.
Joueur	En tant que joueur, je veux accéder au monde afin de progresser dans l'aventure.
Joueur	En tant que joueur, je veux combattre des ennemis afin de gagner de l'expérience.
Joueur	En tant que joueur, je veux utiliser différentes capacités en combat afin d'adapter ma stratégie.
Joueur	En tant que joueur, je veux suivre mon niveau, mon expérience et mon endurance afin de connaître ma progression.
Joueur	En tant que joueur, je veux affronter un boss lorsque j'atteins le niveau requis afin de valider ma progression.
Joueur	En tant que joueur, je veux choisir une faction afin de personnaliser mon parcours.
Administrateur	En tant qu'administrateur, je veux gérer les news afin de maintenir les

4. Diagramme de cas d'utilisation UML

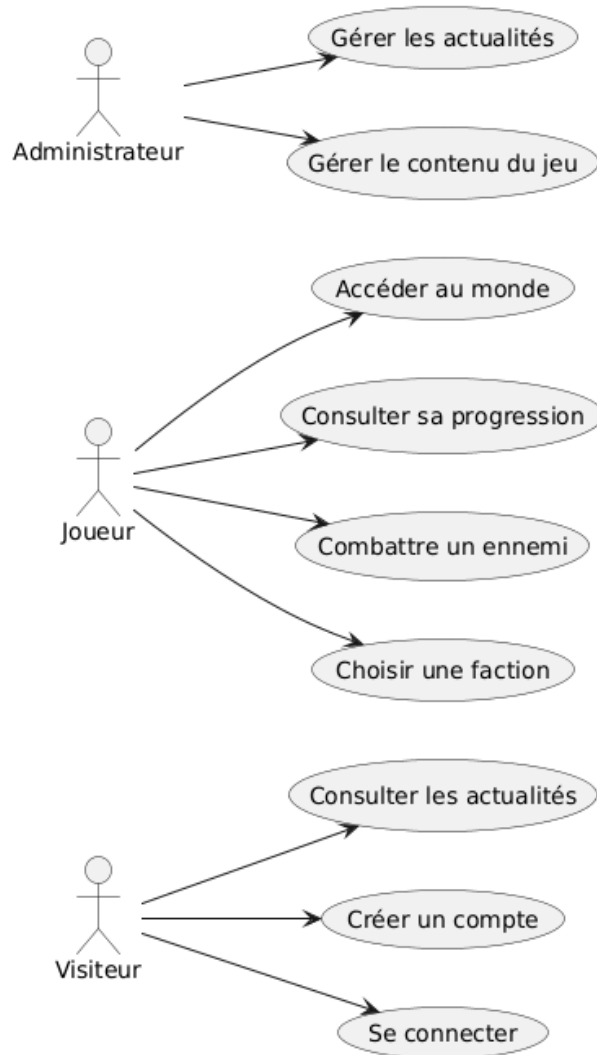


Figure 2 : Diagramme de cas d'utilisation de l'application One Piece Zero

Le diagramme de cas d'utilisation ci-dessus permet de représenter les principales interactions entre les différents acteurs du système et les fonctionnalités de l'application.

Il met notamment en évidence les fonctionnalités accessibles aux visiteurs, aux joueurs authentifiés ainsi qu'aux administrateurs. Ce diagramme permet de visualiser le périmètre fonctionnel du projet et les principales actions réalisables dans l'application.

5. Scénario fonctionnel principal : combat contre un ennemi

Le système de combat constitue l'une des fonctionnalités principales de l'application. Il permet au joueur d'affronter un ennemi dans un système au tour par tour. Cette fonctionnalité met en œuvre plusieurs règles métier : gestion de l'endurance, calcul des dégâts, choix des capacités utilisées, résultat du combat et gain d'expérience. Le cahier des charges prévoit notamment des capacités d'attaque, de soin, de boost, d'esquive, de contre, de renvoi et de paralysie.

Préconditions :

- Le joueur est connecté
- Le joueur possède une progression existante
- Le joueur se trouve dans une zone accessible
- Le joueur dispose d'une endurance suffisante
- Un ennemi est disponible dans la zone

Scénario nominal

- a. Le joueur accède à la zone de combat.
- b. L'application affiche les ennemis disponibles.
- c. Le joueur sélectionne un ennemi.
- d. Le combat est initialisé.
- e. Le joueur choisit une capacité.
- f. Le système vérifie que la capacité est autorisée.
- g. Le système vérifie que le joueur possède assez d'endurance.
- h. Le système applique les effets de la capacité.
- i. L'ennemi effectue son tour.
- j. Le système met à jour les points de vie et l'endurance.
- k. Le combat continue jusqu'à la victoire ou la défaite.
- l. En cas de victoire, le joueur gagne de l'expérience et des récompenses.
- m. La progression est sauvegardée en base de données.

Scénarios alternatifs

Cas	Description
Endurance insuffisante	Si le joueur n'a pas assez d'endurance, la capacité est refusée.
Endurance épuisée	Si le joueur n'a plus d'endurance, le combat ne se lance pas.
Ennemi vaincu	Le combat se termine, les gains sont calculés puis enregistrés

Joueur vaincu	Le combat se termine sans gain.
Accès boss refusé	Si le joueur n'a pas atteint le niveau requis, l'accès au boss est impossible
Données invalides	Si une action ou un identifiant est invalide, le système bloque le traitement

6. Règles métier

Les principales règles métier du projet sont les suivantes :

Règles	Description
Accès aux zones	Une zone peut nécessiter un niveau minimum.
Combat	Un combat nécessite une endurance suffisante.
Capacité	Chaque capacité consomme de l'endurance
Progression	Une victoire permet de gagner de l'expérience, des berries et de la prime.
Montée de niveau	Le niveau du joueur évolue selon son expérience.
Boss Higuma	Le boss est accessible uniquement lorsque le joueur atteint le niveau 6.
Faction	Le choix de faction se fait après le combat de boss Higuma.
Persistance	Les données du joueur, des combats et de la progression sont sauvegardées

Ces règles sont directement issues du périmètre fonctionnel et métier défini dans le cahier des charges.

Spécifications techniques

1. Architecture générale du projet

L'application One Piece Zero repose sur une architecture web en couches permettant de séparer les responsabilités entre l'interface utilisateur, la logique métier et la gestion des données. Cette organisation facilite la maintenance, l'évolution et la sécurisation de l'application.

L'architecture du projet se compose des éléments suivants :

- Un front-end chargé de l'interface utilisateur
- Un back-end développé avec Spring Boot
- Une base de données relationnelle MySQL
- Une communication via API REST utilisant le format JSON

Cette architecture permet d'assurer une séparation claire des responsabilités tout en facilitant les échanges entre les différentes couches de l'application.

2. Technologies utilisées

Les technologies utilisées pour le développement du projet sont les suivantes :

Technologie	Versio n	Utilisation
Java	21	Développement du back-end
Spring Boot	3.5.0	Développement de l'API REST
Maven	3.9	Gestion des dépendances
MySQL	8.4 LTS	Base de données relationnelle
Angular	21.2.17	Framework du front-end
TypeScript	5.9.2	Développement du front-end
Leaflet	1.9.4	Carte interactive du monde

Git	2.49	Gestion de version
GitHub	Cloud	Hébergement du code source
Visual Studio Code	Stable	Environnement de développement
Jira	Cloud	Gestion de projet et suivi des tâches

Le projet a été développé avec Java 21 LTS et Spring Boot 3.5, permettant de bénéficier des fonctionnalités récentes de l'écosystème Java ainsi que d'un framework moderne pour le développement d'applications web sécurisées et maintenables. Le front-end repose sur Angular et TypeScript, offrant une architecture orientée composants facilitant l'organisation du code et la réutilisation des éléments de l'interface.

3. Architecture applicative

Le projet suit une architecture multicouche inspirée du modèle MVC (Modèle – Vue – Contrôleur).

Le front-end de l'application a été développé avec Angular et TypeScript. L'interface utilisateur est organisée sous forme de composants réutilisables permettant de séparer les différentes parties de l'application (authentification, progression du joueur, combats, navigation dans le monde, etc.).

Les services Angular sont utilisés afin de centraliser les appels HTTP vers l'API REST et de faciliter les échanges avec le back-end Spring Boot. Cette organisation permet de limiter le couplage entre les composants, de faciliter la maintenance du projet et d'améliorer la lisibilité du code.

Le routing Angular permet également de gérer la navigation entre les différentes pages de l'application tout en contrôlant l'accès à certaines fonctionnalités selon l'état d'authentification de l'utilisateur.

L'application est organisée en plusieurs couches :

Couche	Description
Controller	Gère les requêtes HTTP reçues par l'application

Service	Implémente la logique métier de l'application
Repository	Assure l'accès aux données et aux opérations de persistance
Entity	Représente les objets persistés dans la base de données
Security	Gère l'authentification, l'autorisation et la sécurisation des routes de l'API
DTO	Transporte les données échangées entre le client et le serveur
Config	Centralise les paramètres et la configuration de l'application

Cette séparation permet :

- Une meilleure maintenabilité
- Une meilleure lisibilité du code
- Une réutilisation des composants
- Une sécurisation plus simple des traitements métier

Le respect de cette architecture facilite également les tests et les évolutions futures du projet.

4. Communication API REST

Le back-end expose une API REST permettant au front-end de communiquer avec l'application. Les échanges reposent principalement sur :

- Le protocole HTTP
- Le format JSON pour le transport des données

Chaque fonctionnalité importante dispose de routes dédiées, par exemple :

- L'authentification
- La gestion des utilisateurs
- La récupération des données du joueur
- La gestion des combats
- La progression du joueur

L'API REST permet une séparation claire entre le front-end et le back-end, rendant l'application plus évolutive et plus modulaire.

5. Gestion des données

Les données de l'application sont stockées dans une base de données relationnelle MySQL. Le projet utilise Spring Data JPA afin de simplifier l'accès aux données, de manipuler les entités Java, de gérer les relations entre les tables et de limiter les requêtes SQL répétitives.

Les principales entités manipulées par l'application sont :

- Utilisateur
- Personnage
- Progression Joueur
- Combat
- Ennemi
- Île
- Zone
- Faction
- Capacité
- Actualité

Le détail complet du modèle relationnel est présenté dans la partie modélisation des données ainsi qu'en annexes.

6. Sécurité technique

La sécurité a été intégrée dès la conception de l'application. L'authentification repose sur Spring Security et l'utilisation de JSON Web Tokens (JWT). Après connexion, l'utilisateur reçoit un jeton d'authentification qu'il doit transmettre lors des appels aux routes protégées de l'API.

Les mots de passe sont stockés sous forme hachée à l'aide de BCrypt afin d'éviter leur stockage en clair. Les données reçues par l'API sont validées avant traitement afin de limiter les erreurs et les données invalides. Les rôles utilisateurs permettent également de contrôler l'accès à certaines fonctionnalités.

7. Organisation du code source

Le code source du projet est organisé de manière à séparer clairement les responsabilités de chaque composant. Le projet est structuré autour des packages principaux suivants :

Package	Contenu
Controller	Gestion des API
Service	Logique métier
Repository	Accès aux données
Entity	Entités JPA
Security	Configuration de sécurité
DTO	Objets de transfert de données
Config	Configuration générale

Cette organisation améliore la lisibilité du projet et facilite son évolution.

8. Schéma d'architecture

Un schéma d'architecture technique a été réalisé afin de représenter les échanges entre le client, l'API REST et la base de données.

Ce schéma présente :

- Le client utilisateur
- Le front-end
- Le serveur Spring Boot
- Les différentes couches applicatives
- La base de données MySQL
- Les mécanismes de sécurité
- Les échanges HTTP/JSON

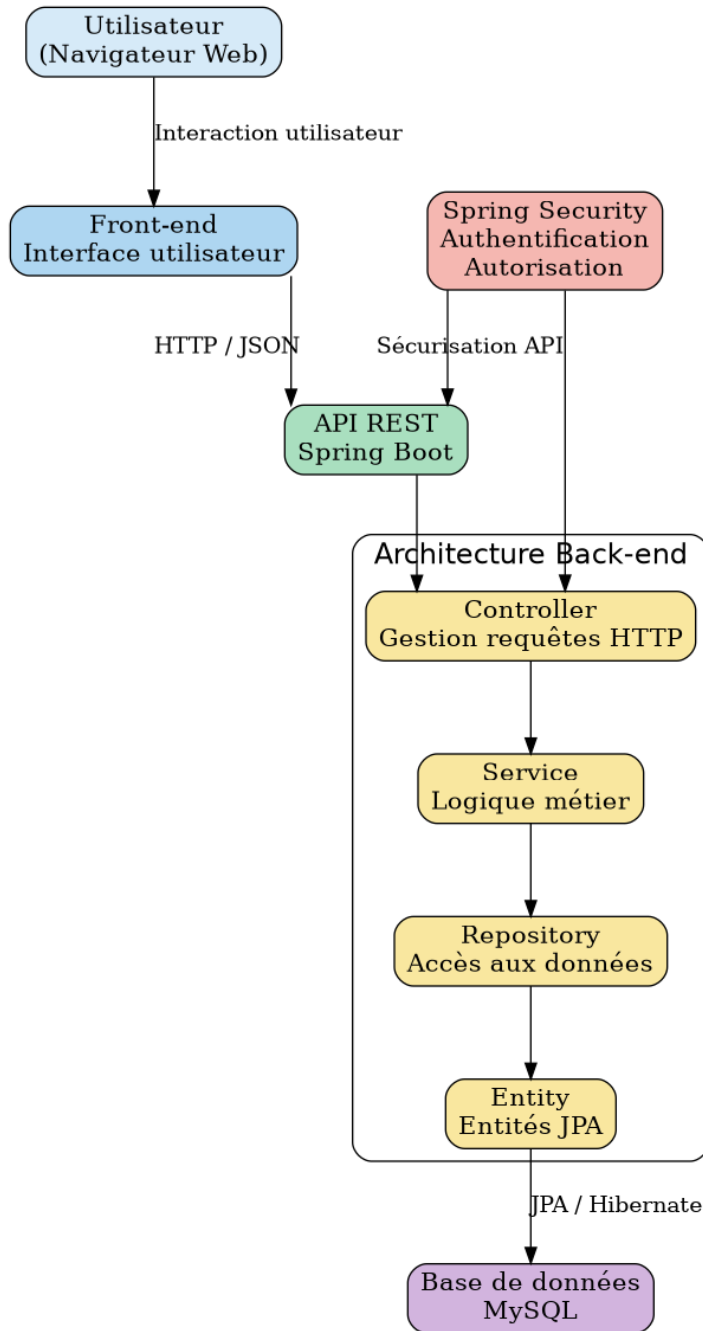


Figure 3 : Architecture technique de l'application One Piece Zero

L'architecture de l'application repose sur une séparation claire entre le client, le back-end et la base de données.

Le joueur interagit avec l'application via son navigateur web. Les actions réalisées par l'utilisateur, comme la connexion, l'accès au monde ou le lancement d'un combat, sont envoyées au serveur à travers des requêtes HTTP.

Le front-end communique avec le back-end grâce à une API REST développée avec Spring Boot. Les données échangées utilisent le format JSON afin de faciliter les communications entre les différentes couches de l'application.

Lorsqu'une action est effectuée par le joueur, par exemple un combat, la requête est d'abord traitée par le Controller concerné. Celui-ci transmet ensuite les informations au Service correspondant afin d'exécuter la logique métier du combat (calcul des dégâts, gestion de l'endurance, validation des capacités, gain d'expérience, etc.).

Les données nécessaires sont récupérées ou enregistrées via les Repositories utilisant Spring Data JPA et Hibernate. Ces composants assurent la communication avec la base de données MySQL.

La sécurité de l'application est gérée grâce à Spring Security. Ce composant permet notamment :

- La sécurisation des routes API
- Le contrôle des accès
- La gestion des rôles
- La protection des données utilisateurs

Cette architecture permet d'obtenir une application :

- Maintenable
- Modulaire
- Évolutive
- Plus simple à tester et à sécuriser

Réalisations du candidat

1. Conception de l'application

Conception de l'interface utilisateur

Avant le développement de l'interface, des wireframes puis des maquettes ont été réalisés afin de préparer l'organisation des principaux écrans de l'application. Cette étape a permis de représenter le parcours utilisateur du MVP, depuis l'arrivée sur la page d'accueil jusqu'à l'accès à l'espace joueur, l'exploration du monde et le combat.

Les maquettes ont été pensées en cohérence avec les besoins fonctionnels identifiés précédemment : consultation des actualités, inscription, connexion, suivi de la progression, accès aux zones, choix des actions de combat et gestion de la session utilisateur.

L'objectif de ces wireframes n'était pas de figer le design définitif de l'application, mais de valider la structure générale des interfaces, la hiérarchie des informations affichées et les principales interactions proposées au joueur. Ils ont également servi de support pour anticiper les composants Angular nécessaires au développement du front-end.

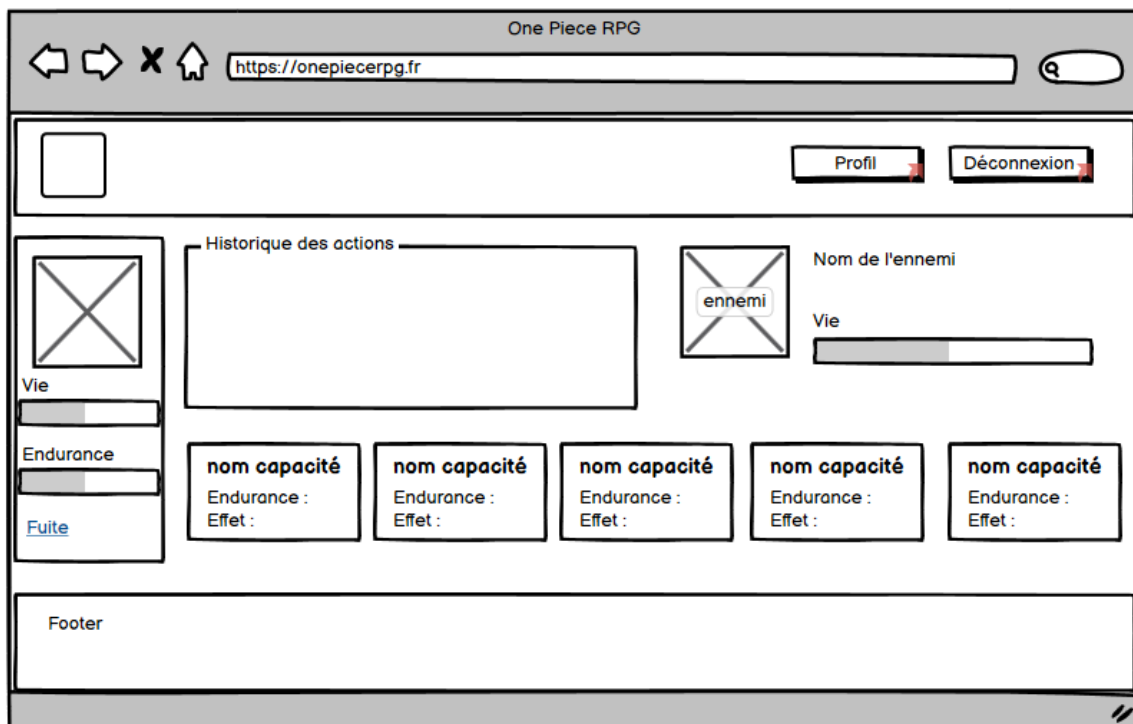


Figure 4 : Wireframe de l'interface de combat

Cette maquette illustre l'évolution du wireframe vers une interface plus proche du rendu final. Elle reprend les éléments identifiés lors de la conception : affichage des statistiques du joueur, informations de l'ennemi, historique des actions et choix des capacités disponibles pendant le combat.



Figure 5 : Maquette de l'interface de combat

Modélisation des données

Après la définition du périmètre fonctionnel et la conception des principales interfaces, une phase de modélisation des données a été réalisée afin de structurer les informations nécessaires au fonctionnement de l'application.

Cette modélisation avait pour objectif d'identifier les principales entités métier du projet, leurs attributs ainsi que les relations existantes entre elles. Elle permet notamment de représenter les utilisateurs, les personnages, la progression du joueur, les îles, les zones, les ennemis, les combats, les capacités, les factions ainsi que les actualités.

Un Modèle Conceptuel de Données a d'abord été réalisé afin de représenter l'organisation générale des données indépendamment de leur implémentation technique. Ce MCD a permis de vérifier la cohérence du modèle métier avec les règles définies dans le cahier des charges, notamment la progression du joueur, l'accès aux zones, la gestion des combats et le choix de faction.

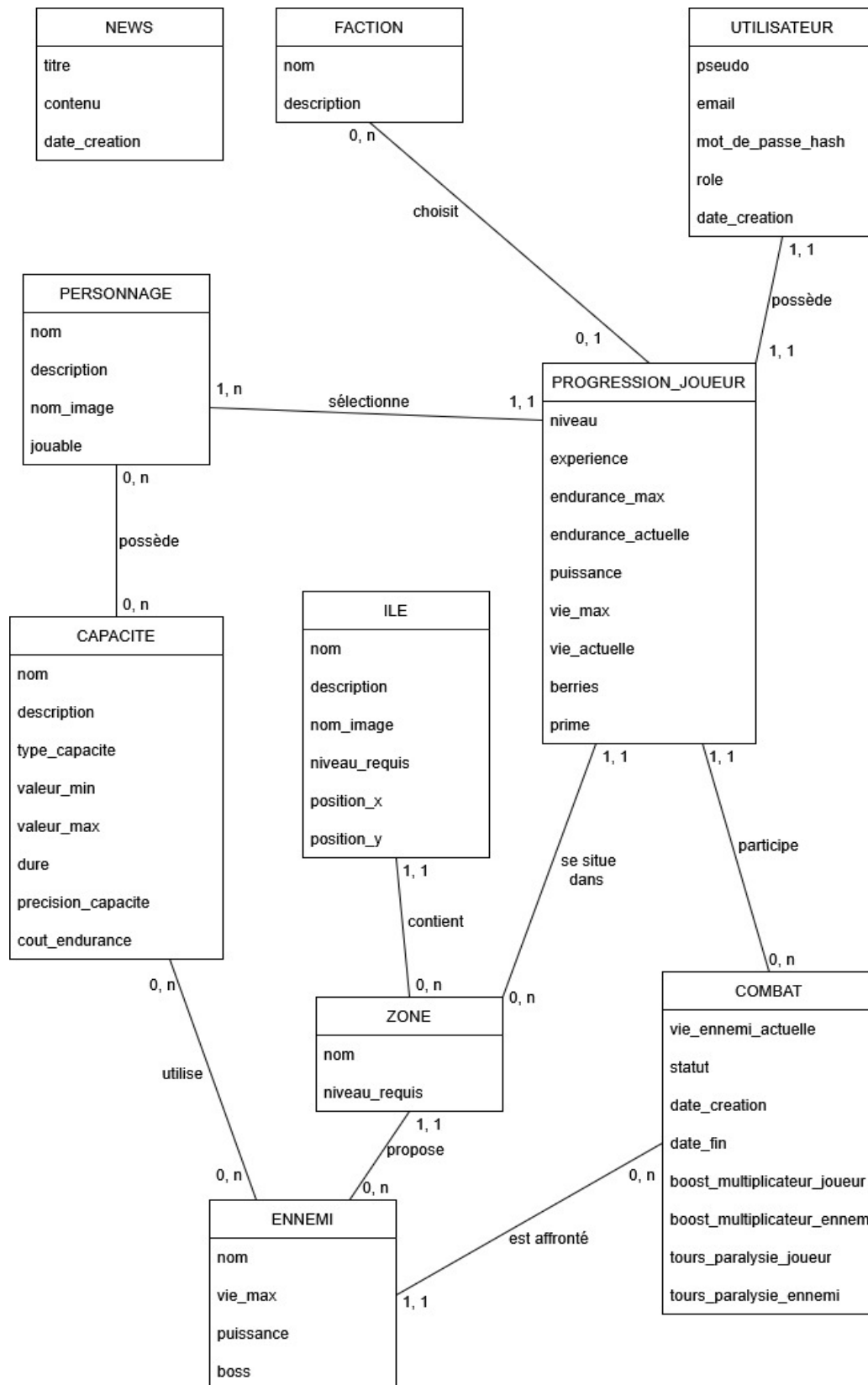


Figure 6 : Modèle Conceptuel de Données de l'application One Piece Zero

Le Modèle Physique de Données a ensuite été conçu afin de traduire cette modélisation en structure relationnelle exploitable par MySQL. Il précise les tables, les clés primaires, les clés étrangères ainsi que les relations entre les différentes données persistées.

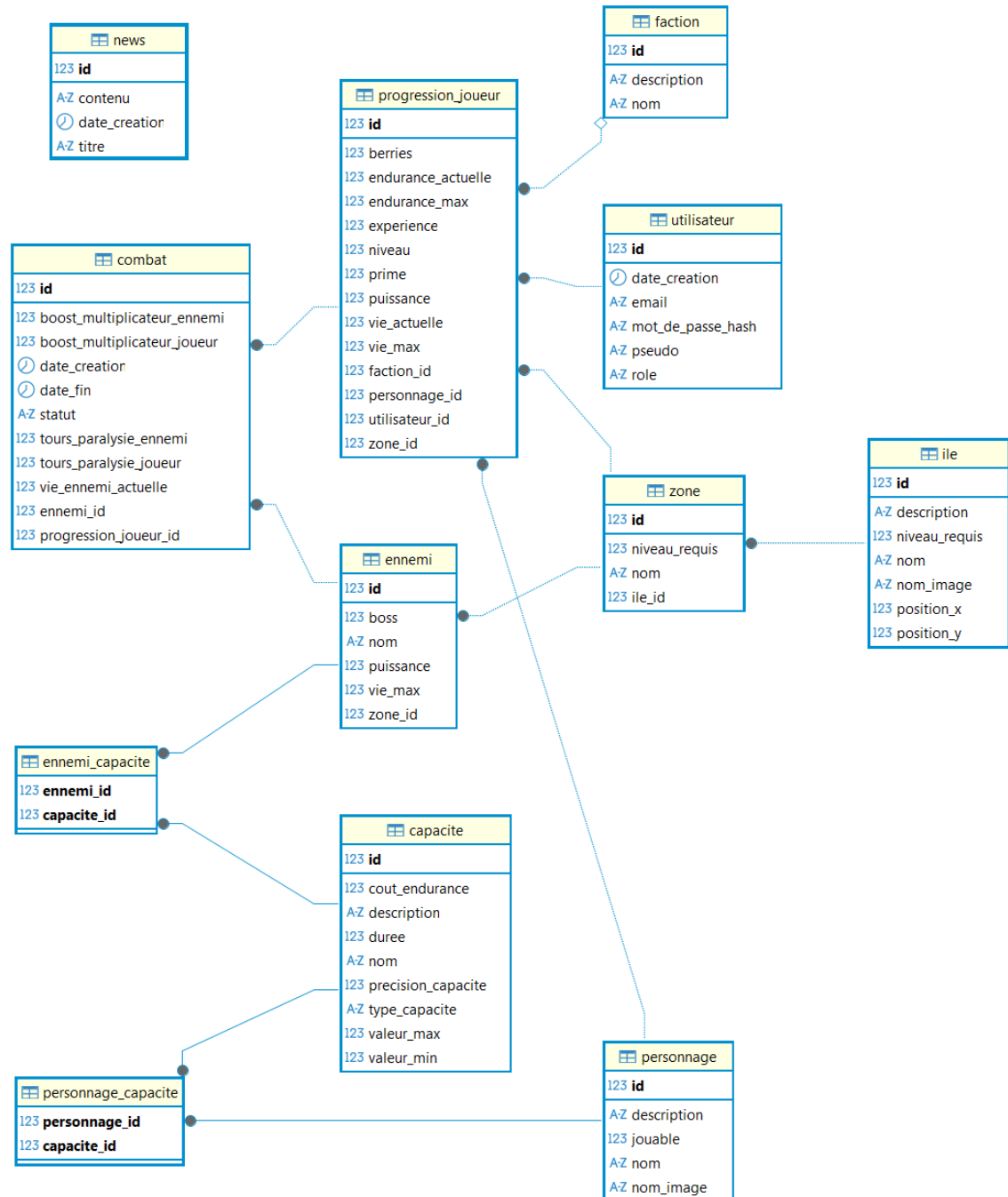


Figure 7 : Modèle Physique de Données de l'application One Piece Zero

2. Développement du back-end

Architecture du back-end

Le développement du back-end de l'application a été réalisé à l'aide du framework Spring Boot. Cette technologie a été retenue afin de bénéficier d'un environnement de développement moderne, facilitant la création d'API REST, la gestion des dépendances, la persistance des données ainsi que la sécurisation de l'application.

L'architecture retenue repose sur une séparation des responsabilités entre plusieurs couches applicatives. Cette organisation permet de limiter le couplage entre les différents composants, de faciliter la maintenance du projet et de rendre les évolutions futures plus simples à mettre en œuvre.

Le projet est principalement organisé autour des packages suivants :

- Controller : gestion des requêtes HTTP et des réponses de l'API.
- Service : implémentation de la logique métier de l'application.
- Repository : accès aux données et communication avec la base de données.
- Entity : représentation des entités persistées.
- DTO : objets de transfert de données utilisés par l'API.
- Security : configuration des mécanismes de sécurité.
- Config : configuration générale de l'application.

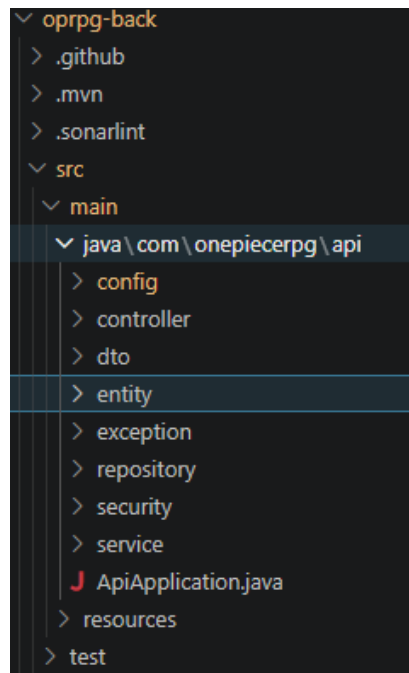


Figure 8 : Organisation des packages du projet Spring Boot

Les contrôleurs constituent le point d'entrée de l'application. Ils reçoivent les requêtes envoyées par le front-end Angular et transmettent les traitements aux services correspondants.

Les services centralisent la logique métier de l'application. Ils permettent notamment de gérer l'authentification des utilisateurs, la progression des joueurs, les combats, les capacités ou encore les règles de progression.

L'accès aux données est assuré par les repositories Spring Data JPA. Cette couche permet de manipuler les entités persistées sans avoir à écrire manuellement la majorité des requêtes SQL.

Cette architecture favorise la lisibilité du code, la réutilisation des composants et la réalisation des tests unitaires. Elle permet également de faire évoluer l'application en limitant l'impact des modifications sur les autres couches du projet.

Gestion des utilisateurs

La gestion des utilisateurs constitue la première fonctionnalité développée dans l'application. Elle permet à un visiteur de créer un compte, de s'authentifier et d'accéder à son espace de jeu.

L'API expose plusieurs routes dédiées à l'inscription et à la connexion des utilisateurs. Ces routes sont accessibles sans authentification afin de permettre la création d'un compte et l'obtention d'un jeton d'accès.

Lors de l'inscription, plusieurs vérifications sont réalisées afin de garantir la cohérence des données :

- Vérification de l'unicité du pseudo
- Vérification de l'unicité de l'adresse électronique
- Validation des données reçues
- Hachage du mot de passe avant son enregistrement

Les mots de passe ne sont jamais stockés en clair dans la base de données. Ils sont hachés à l'aide de l'algorithme BCrypt avant d'être enregistrés.

L'inscription d'un nouvel utilisateur entraîne également la création automatique de sa progression initiale. Le joueur débute ainsi l'aventure avec le personnage de départ, un niveau initial, des points de vie et une endurance définis ainsi qu'une position de départ dans le monde du jeu.

La connexion permet ensuite de vérifier les informations d'authentification fournies par l'utilisateur. En cas de succès, l'application génère un jeton JWT permettant d'accéder aux routes protégées de l'API.

Cette organisation permet de séparer clairement les responsabilités entre les contrôleurs, chargés de gérer les requêtes HTTP, et les services, qui implémentent les règles métier liées à l'authentification et à la création des données du joueur.

L'image suivante présente le processus d'inscription et de connexion mis en œuvre dans l'application. Lors de l'inscription, le mot de passe est haché avant son enregistrement en base de données. Lors de la connexion, un jeton JWT est généré puis renvoyé au front-end afin de permettre l'accès aux routes protégées de l'API.

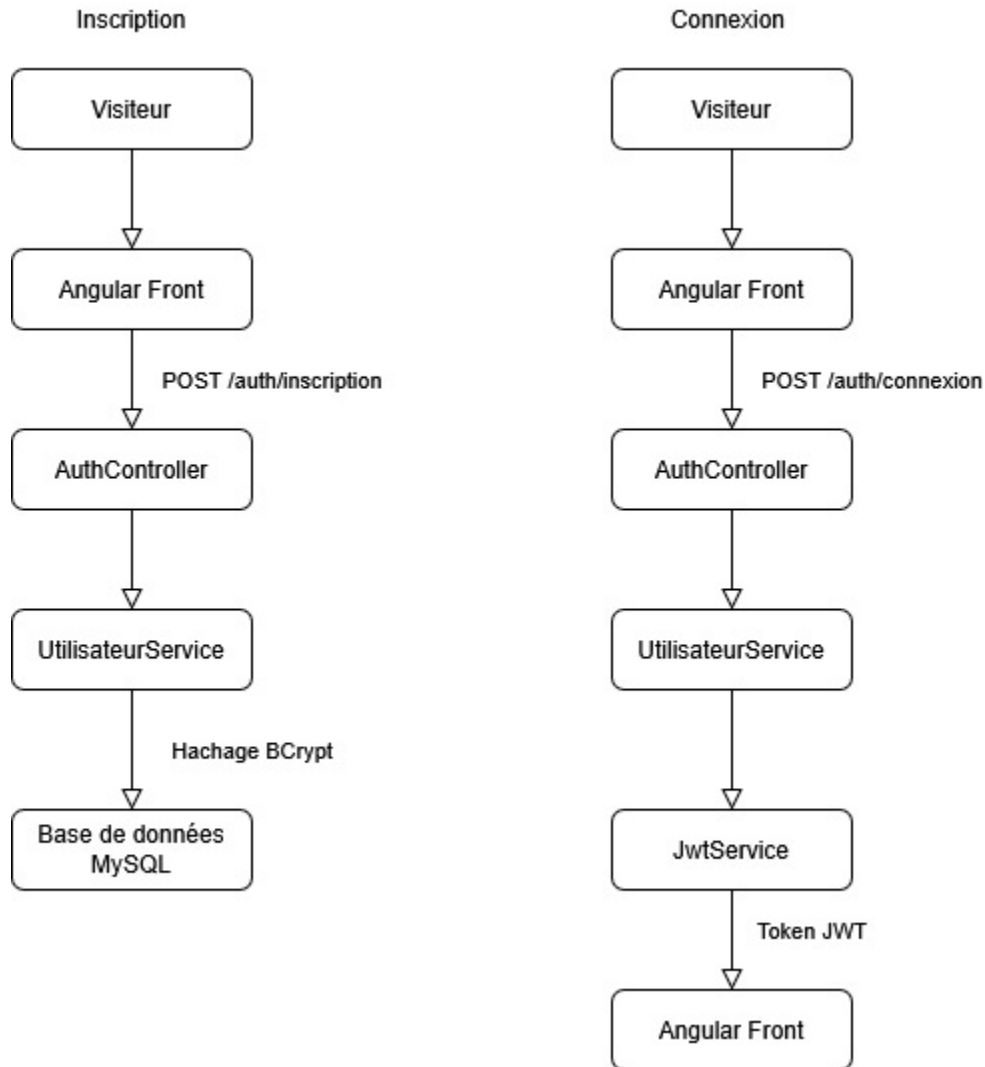


Figure 9 : Processus d'inscription et d'authentification

```

public UtilisateurResponseDto creerUtilisateur(InscriptionRequest request) {
    utilisateurRepository.findByEmail(request.getEmail())
        .ifPresent(utilisateur -> {
            throw new IllegalArgumentException("Cet email existe déjà");
        });

    utilisateurRepository.findByPseudo(request.getPseudo())
        .ifPresent(utilisateur -> {
            throw new IllegalArgumentException("Ce pseudo existe déjà");
        });

    Utilisateur utilisateur = new Utilisateur();
    utilisateur.setPseudo(request.getPseudo());
    utilisateur.setEmail(request.getEmail());
    utilisateur.setMotDePasseHash(passwordEncoder.encode(request.getMotDePasse()));
    utilisateur.setRole("USER");

    Utilisateur utilisateurSauvegarde = utilisateurRepository.save(utilisateur);
    progressionJoueurService.creerProgressionInitiale(utilisateurSauvegarde);

    return convertirEnDto(utilisateurSauvegarde);
}

```

Extrait de code 1 : Création d'un utilisateur et initialisation de sa progression

Cet extrait présente la création d'un nouvel utilisateur lors de l'inscription. Plusieurs vérifications sont effectuées afin de garantir l'unicité du pseudo et de l'adresse électronique. Le mot de passe est ensuite haché à l'aide de BCrypt avant l'enregistrement en base de données. Une fois l'utilisateur créé, une progression initiale est automatiquement générée afin de permettre au joueur de débiter l'aventure.

Gestion de la progression du joueur

La progression du joueur constitue l'élément central du système de jeu. Elle permet de conserver l'ensemble des informations liées à l'aventure du joueur, telles que son niveau, son expérience, son endurance, sa puissance ou encore sa position dans le monde.

Lors de la création d'un compte, une progression initiale est automatiquement générée. Le joueur débute l'aventure avec le personnage de départ, un niveau initial ainsi qu'une zone de départ située sur l'île de Dawn.

Les informations de progression sont enregistrées en base de données afin d'assurer la persistance des données entre les différentes sessions de jeu. Le joueur retrouve ainsi son personnage dans l'état où il a quitté l'application.

La progression gère également l'évolution du joueur au cours de l'aventure. Les combats permettent de gagner de l'expérience et des récompenses, tandis que certaines actions, comme le repos ou la consommation de lait, permettent de récupérer l'endurance ou les points de vie.

La position du joueur dans le monde est également conservée. Chaque déplacement vers une nouvelle zone entraîne une mise à jour de sa progression, permettant ainsi de suivre son évolution dans l'univers du jeu.

Cette centralisation des données facilite la gestion des règles métier et permet d'éviter la duplication des informations au sein de l'application.

```
public ProgressionJoueur creerProgressionInitiale(Utilisateur utilisateur) {
    Personnage personnage = personnageRepository.findByNom(NOM_PERSONNAGE_DEPART)
        .orElseThrow(() -> new RessourceIntrouvableException("Personnage de départ non trouvé"));

    Zone zone = zoneRepository.findByNom(NOM_ZONE_DEPART)
        .orElseThrow(() -> new RessourceIntrouvableException("Zone de départ non trouvée"));

    ProgressionJoueur progression = new ProgressionJoueur();
    progression.setUtilisateur(utilisateur);
    progression.setPersonnage(personnage);
    progression.setZone(zone);
    progression.setVieActuelle(progression.getVieMax());
    progression.setEnduranceActuelle(progression.getEnduranceMax());

    return progressionJoueurRepository.save(progression);
}
```

Extrait de code 2 : Création de la progression initiale du joueur

Cette méthode permet d'initialiser la progression d'un nouveau joueur. Elle associe l'utilisateur au personnage de départ ainsi qu'à la zone de départ du jeu. Les statistiques initiales sont également mises en place afin de garantir un état cohérent dès la première connexion. Cette automatisation simplifie la création d'un nouveau compte et assure une expérience identique pour tous les joueurs.

Développement du système de combat

Le système de combat constitue la fonctionnalité métier la plus importante du projet. Il permet au joueur d'affronter un ennemi dans un combat en solo au tour par tour.

Lorsqu'un combat est lancé, l'application vérifie d'abord que le joueur dispose d'une progression existante, qu'il se trouve dans une zone valide et qu'un ennemi peut être affronté. Un combat est ensuite créé et enregistré en base de données avec les informations nécessaires à son déroulement, comme l'ennemi affronté, ses points de vie actuels, le statut du combat et les éventuels effets actifs.

Le joueur peut ensuite utiliser différentes capacités. Chaque capacité possède un type, un coût en endurance, une valeur d'effet et parfois une durée. Cette organisation permet de gérer plusieurs comportements différents sans devoir créer une logique spécifique pour chaque action.

Les capacités disponibles permettent notamment :

- D'infliger des dégâts
- De récupérer des points de vie
- D'augmenter temporairement les dégâts
- D'esquiver une attaque
- De contrer l'adversaire
- De renvoyer une attaque

- De paralyser l'ennemi

À chaque tour, le système vérifie que l'action choisie est valide et que le joueur possède suffisamment d'endurance. Si ces conditions sont respectées, les effets de la capacité sont appliqués, puis l'ennemi effectue à son tour une action.

Le combat se poursuit jusqu'à ce que le joueur ou l'ennemi n'ait plus de points de vie. En cas de victoire, le joueur gagne de l'expérience et de la prime. Sa progression est ensuite mise à jour et sauvegardée en base de données.

Cette fonctionnalité met en œuvre plusieurs règles métier importantes : gestion des points de vie, gestion de l'endurance, effets temporaires, calcul des dégâts, victoire, défaite et mise à jour de la progression du joueur.

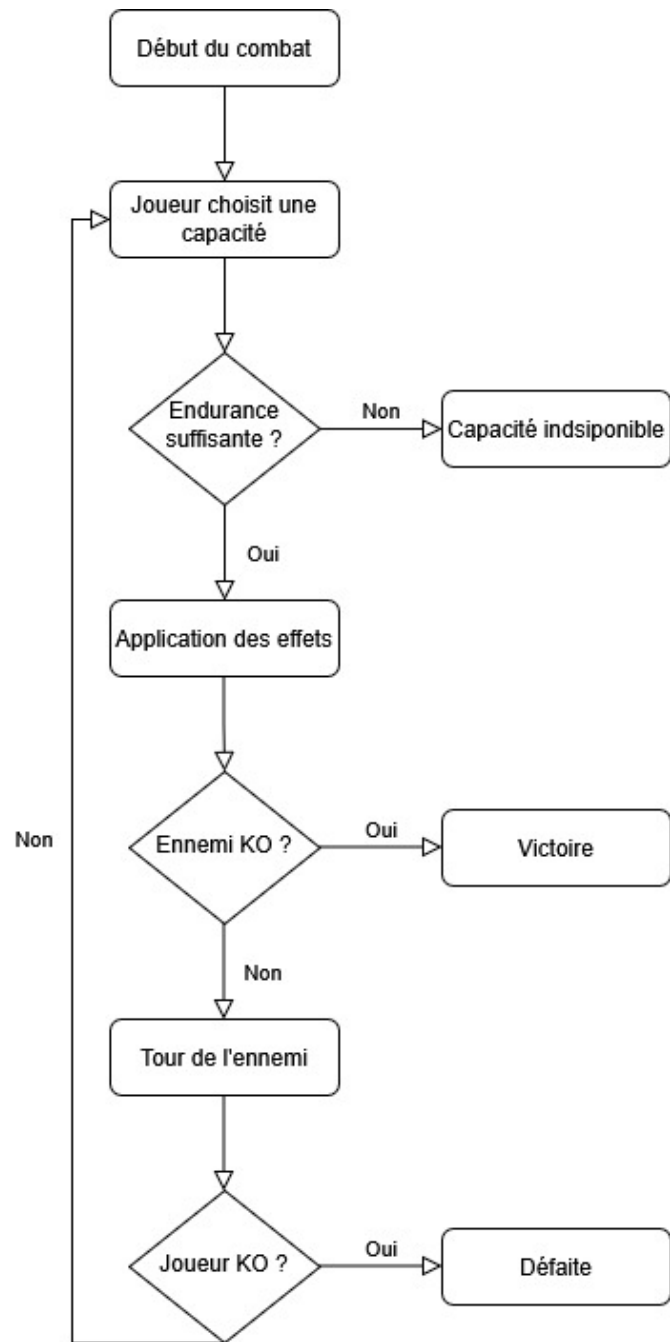


Figure 10 : Déroulement d'un combat

```

public CombatResponse utiliserCapacite(Long capaciteId) {
    ProgressionJoueur progression = recupererProgressionConnectee();
    Combat combat = recupererCombatEnCours(progression);
    Capacite capaciteJoueur = recupererCapaciteJoueur(progression, capaciteId);

    verifierCoutEndurance(progression, capaciteJoueur);
    verifierNonParalyse(combat);

    TourResultat actionJoueur = calculerAction(
        capaciteJoueur, progression.getPuissance(), combat.getBoostMultiplieurJoueur());
    TourResultat actionEnnemi = calculerActionEnnemi(combat);

    consommerEndurance(progression, capaciteJoueur);

    appliquerActionSurEnnemi(actionJoueur, actionEnnemi, combat);
    appliquerActionSurJoueur(actionEnnemi, actionJoueur, combat, progression);

    RecompenseCombatResponse recompense = verifierFinCombat(combat, progression);

    combatRepository.save(combat);
    progressionJoueurRepository.save(progression);

    return convertir(combat, recompense);
}

```

Extrait de code 3 : Déroulement d'un tour de combat

Cet extrait illustre la méthode principale utilisée lors d'un tour de combat. Le système récupère la progression du joueur et le combat en cours, vérifie les conditions d'utilisation de la capacité choisie puis applique les différents effets du tour. Les actions du joueur et de l'ennemi sont ensuite résolues afin de déterminer le résultat du combat. Cette méthode centralise ainsi une grande partie des règles métier de l'application.

La mise en œuvre de cette logique métier a constitué l'une des principales difficultés du projet, notamment en raison des interactions entre les capacités, les effets temporaires, l'endurance et les conditions de victoire ou de défaite.

3. Sécurisation de l'application

Authentification et contrôle des accès

La sécurité a constitué un élément important du développement de l'application. Dès la conception du projet, plusieurs mécanismes ont été mis en place afin de protéger les données des utilisateurs et de sécuriser l'accès aux fonctionnalités du jeu.

L'authentification repose sur Spring Security associé à l'utilisation de JSON Web Tokens (JWT). Lors de la connexion, les informations d'identification de l'utilisateur sont vérifiées par l'application.

Si les informations fournies sont valides, un jeton JWT est généré puis transmis au front-end. Ce jeton est ensuite envoyé lors des appels aux routes protégées de l'API à l'aide de l'en-tête HTTP Authorization.

À chaque requête, le serveur vérifie la validité du jeton avant d'autoriser l'accès aux ressources demandées.

Cette approche présente plusieurs avantages :

- Une authentification sans session serveur
- Une meilleure séparation entre le front-end et le back-end
- Une sécurisation des routes de l'API
- Une limitation des accès non autorisés

Les mots de passe ne sont jamais stockés en clair dans la base de données. Ils sont hachés à l'aide de l'algorithme BCrypt avant leur enregistrement. L'application utilise également des rôles utilisateurs permettant de différencier les fonctionnalités accessibles aux joueurs et aux administrateurs.

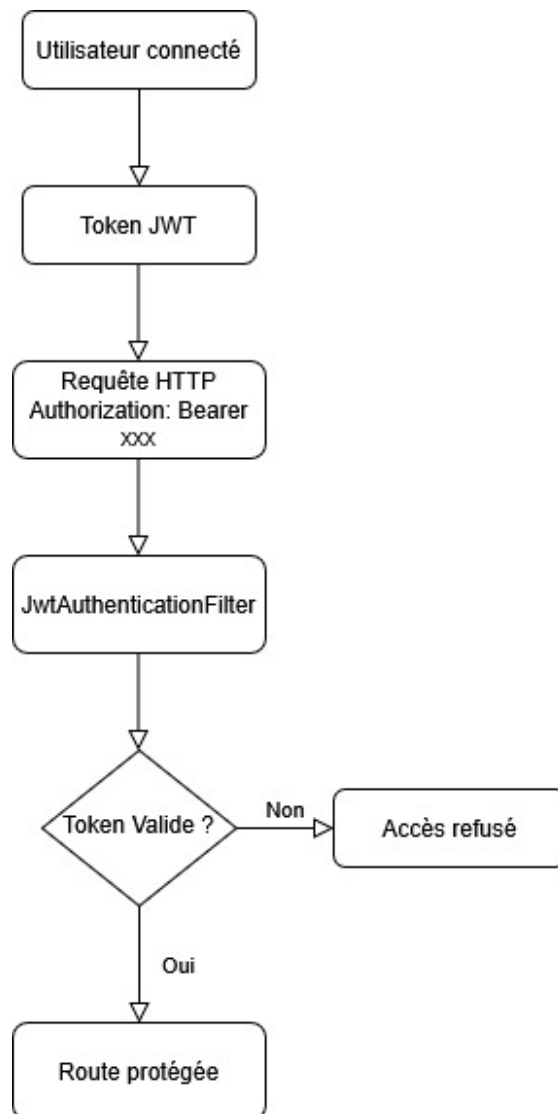


Figure 11 : Vérification d'une requête authentifiée

```
String authorizationHeader = request.getHeader("Authorization");
if (authorizationHeader == null || !authorizationHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}
String token = authorizationHeader.substring(7);
String email = jwtService.extraireEmail(token);
```

Extrait de code 4 : Vérification du jeton JWT

Cet extrait présente le fonctionnement du filtre JWT utilisé par l'application. À chaque requête protégée, le serveur récupère le jeton transmis dans l'en-tête HTTP. Le jeton est ensuite analysé afin d'identifier l'utilisateur et de vérifier sa validité avant d'autoriser l'accès aux ressources sécurisées. Ce mécanisme permet de protéger l'API tout en conservant une architecture sans session.

Validation des données

Afin de garantir la cohérence des données manipulées par l'application, plusieurs contrôles sont réalisés avant les traitements métier.

Les données reçues par l'API sont validées afin d'éviter les valeurs invalides ou les comportements incohérents. Des vérifications sont notamment réalisées sur les informations d'inscription, les capacités utilisées pendant les combats ou encore les conditions d'accès aux différentes fonctionnalités.

Ces contrôles permettent d'améliorer la robustesse de l'application et de limiter les erreurs liées aux données utilisateur.

4. Développement du front-end

Organisation du front-end

Le front-end de l'application a été développé avec Angular et TypeScript. Il constitue l'interface utilisée par le joueur pour accéder aux différentes fonctionnalités du jeu : consultation des actualités, inscription, connexion, accès au monde, exploration des îles, gestion de la progression et combats.

L'application est organisée autour de plusieurs composants Angular représentant les principales pages et fonctionnalités du projet. Cette organisation permet de séparer les différentes parties de l'interface et de faciliter leur maintenance.

Les services Angular sont utilisés afin de centraliser les appels vers l'API REST. Ils permettent notamment de récupérer les informations du joueur, de lancer un combat, d'utiliser une capacité, de gérer l'authentification ou encore de charger les données liées au monde du jeu.

Le routing Angular permet de gérer la navigation entre les différentes pages de l'application. Certaines routes sont protégées afin d'empêcher un utilisateur non connecté d'accéder aux pages réservées aux joueurs authentifiés.

Cette organisation permet de conserver un front-end structuré, lisible et évolutif, tout en séparant clairement l'affichage, la navigation et les échanges avec le back-end.

Authentification côté front-end

La gestion de l'authentification côté front-end repose sur le jeton JWT renvoyé par le back-end lors de la connexion. Une fois reçu, ce jeton est conservé par l'application afin d'être transmis lors des appels aux routes protégées de l'API.

Un interceptor Angular permet d'ajouter automatiquement le jeton JWT dans l'en-tête HTTP des requêtes envoyées au back-end. Cela évite de répéter ce traitement dans chaque service et permet de centraliser la gestion de l'authentification.

Des guards sont également utilisés afin de contrôler l'accès à certaines pages. Par exemple, un utilisateur non connecté ne peut pas accéder à son espace joueur, au monde ou aux combats. Ce contrôle côté front-end vient compléter la sécurisation réalisée côté back-end avec Spring Security.

Cette double vérification permet d'améliorer l'expérience utilisateur tout en renforçant la cohérence globale de la sécurité de l'application.

Carte interactive du monde

L'une des fonctionnalités principales du front-end est l'affichage du monde du jeu sous forme de carte interactive. Cette carte a été réalisée à l'aide de la bibliothèque Leaflet.

Leaflet permet d'afficher une image du monde et de positionner les îles disponibles à des coordonnées précises. Chaque île peut ainsi être représentée visuellement sur la carte et devenir accessible selon la progression du joueur.

Lorsqu'un joueur sélectionne une île disponible, il peut accéder aux zones associées et poursuivre son aventure. Les îles ou zones non accessibles peuvent être bloquées selon le niveau requis, ce qui permet de respecter les règles métier définies dans le cahier des charges.



Figure 12 : Écran du monde par défaut

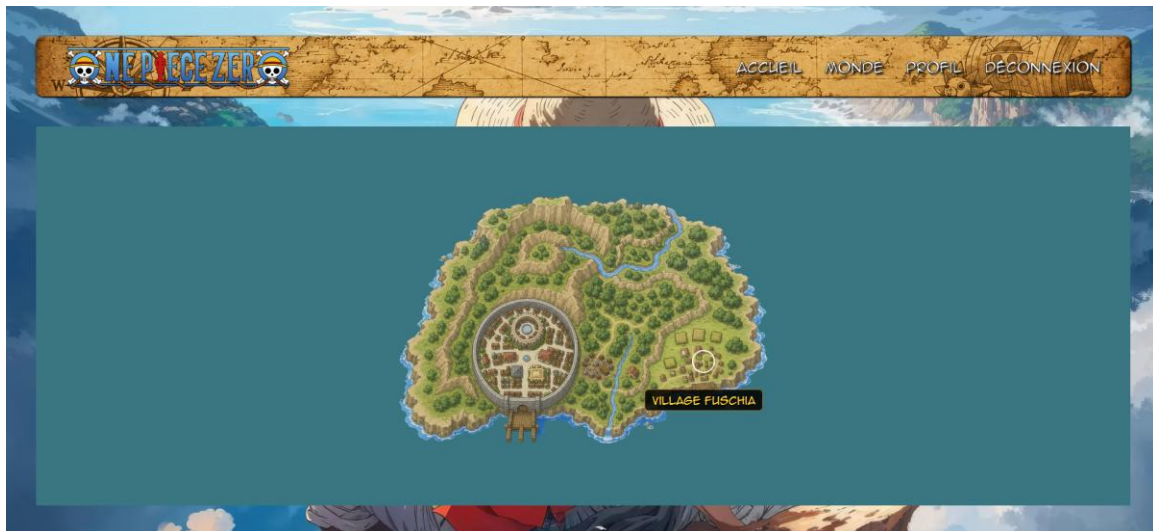


Figure 13 : Écran du monde lorsqu'une île est sélectionnée

Cette fonctionnalité permet de rendre l'exploration plus visuelle et plus immersive, tout en conservant une logique de progression contrôlée par le back-end.

Interface de combat

L'interface de combat a été conçue afin de présenter clairement les informations nécessaires au joueur pendant un affrontement. Elle affiche notamment les points de vie du joueur, son endurance, les informations de l'ennemi, l'historique des actions ainsi que les capacités disponibles.

Lorsqu'une capacité est utilisée, le front-end envoie une requête à l'API REST afin que le back-end applique les règles métier du combat. Le résultat du tour est ensuite renvoyé au front-end et l'interface est mise à jour avec les nouvelles valeurs.

Cette organisation permet de conserver la logique métier côté back-end, tout en utilisant Angular pour afficher dynamiquement l'évolution du combat. Le front-end ne calcule pas directement les dégâts ou les effets des capacités : il se contente d'afficher les résultats transmis par l'API.

Ce choix permet de garantir la cohérence du système de combat et d'éviter que des règles importantes soient uniquement gérées côté client.

5. Déploiement et configuration de l'application

Configuration de l'environnement de développement

Le développement de l'application a nécessité la mise en place d'un environnement permettant d'exécuter simultanément le front-end Angular, le back-end Spring Boot ainsi que la base de données MySQL.

Le back-end est exécuté à l'aide de Spring Boot et Maven tandis que le front-end Angular est lancé indépendamment grâce aux outils de développement fournis par le framework. Cette séparation permet de développer et de tester les différentes parties de l'application de manière autonome.

L'utilisation de Git et GitHub a également permis de suivre l'évolution du projet et de conserver un historique des différentes fonctionnalités développées.

Configuration de l'application

Plusieurs paramètres de configuration sont nécessaires au fonctionnement de l'application.

Le back-end utilise des fichiers de configuration Spring Boot permettant notamment de définir :

- La connexion à la base de données
- Les paramètres de sécurité
- La clé utilisée pour les jetons JWT
- Les paramètres de l'application

Cette organisation permet d'isoler les éléments de configuration du code métier et facilite les évolutions futures du projet.

Perspectives de déploiement

Une solution de conteneurisation basée sur Docker a été étudiée afin de faciliter le déploiement futur de l'application. Cette approche permettrait d'isoler le front-end, le

back-end ainsi que la base de données dans des conteneurs distincts afin de simplifier l'installation et la mise en production de l'application.

Bien que cette solution ne soit pas encore utilisée dans la version actuelle du projet, elle constitue une piste d'évolution intéressante pour les futures versions du projet.

6. Tests et validation de l'application

Validation de l'API avec Swagger

Afin de faciliter le développement et la validation des différentes fonctionnalités du back-end, l'outil Swagger a été intégré à l'application. Swagger permet de générer automatiquement une documentation interactive de l'API REST à partir des contrôleurs Spring Boot. Cette interface offre la possibilité de consulter les routes disponibles, les paramètres attendus, les données retournées ainsi que les différents codes HTTP.

L'outil a été utilisé tout au long du développement afin de tester manuellement les fonctionnalités avant leur intégration au front-end Angular.

Les principales routes vérifiées concernent :

- L'inscription des utilisateurs
- La connexion et la génération des jetons JWT
- La récupération de la progression du joueur
- Le lancement des combats
- L'utilisation des capacités
- La consultation des données du jeu

Cette approche a permis de détecter rapidement certaines anomalies et de valider le bon fonctionnement des échanges entre le client et le serveur.



Figure 14 : Authentification d'un utilisateur avec Swagger

L'image ci-dessus présente l'exécution d'une requête de connexion depuis l'interface Swagger. Après validation des identifiants, l'API retourne un code HTTP 200

ainsi qu'un jeton JWT permettant d'accéder aux routes protégées de l'application. Cette validation a permis de vérifier le bon fonctionnement du mécanisme d'authentification et de la génération des jetons de sécurité.

Tests unitaires

Les tests unitaires ont été réalisés afin de vérifier le fonctionnement des différents services de l'application de manière isolée.

L'utilisation de JUnit 5 associée à Mockito permet de simuler les dépendances des services afin de tester uniquement la logique métier de chaque composant.

Cette approche facilite l'identification des erreurs et permet de sécuriser les évolutions futures du projet.

Plusieurs services ont notamment fait l'objet de tests :

- Le service de gestion des utilisateurs
- Le service de progression du joueur
- Le service d'authentification JWT
- Les services liés au système de combat

Ces tests permettent de vérifier les règles métier sans nécessiter le démarrage complet de l'application.

Tests des contrôleurs REST

Des tests ont également été réalisés sur les contrôleurs REST afin de vérifier le bon fonctionnement des échanges HTTP entre le client et l'application.

L'outil MockMvc permet de simuler des requêtes HTTP sans démarrer un serveur web complet. Cette approche facilite la validation des réponses produites par les contrôleurs.

Les tests permettent notamment de vérifier :

- Les codes de retour HTTP
- Les données renvoyées par l'API
- Les cas d'erreur
- Les contrôles de sécurité

Cette validation garantit le respect du contrat d'interface entre le front-end et le back-end.

```
@Test
void shouldRegisterUser() throws Exception {
    mockMvc.perform(post("/api/auth/inscription")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isCreated());
}
```

Extrait de code 5 : Test d'inscription d'un utilisateur

Cet extrait présente un test réalisé sur le contrôleur d'authentification. Il permet de vérifier que l'inscription d'un utilisateur renvoie bien le code HTTP attendu lorsque les données fournies sont valides.

Ces tests permettent de garantir la stabilité des routes exposées par l'API.

Tests d'intégration

Des tests d'intégration ont également été mis en place afin de vérifier le bon fonctionnement des interactions entre les différentes couches de l'application.

Ces tests permettent notamment de valider les échanges avec la base de données, les requêtes JPA ainsi que la cohérence des données enregistrées.

Une base de données H2 est utilisée pendant l'exécution des tests afin d'isoler l'environnement de test de la base de données principale.

Cette approche permet de reproduire le comportement réel de l'application tout en conservant des temps d'exécution réduits.

```

@DataJpaTest
class UtilisateurRepositoryTest {

    @Autowired
    private UtilisateurRepository utilisateurRepository;

    @Test
    void shouldFindUtilisateurByEmail() {
        Utilisateur utilisateur = new Utilisateur();
        utilisateur.setPseudo("luffy");
        utilisateur.setEmail("luffy@test.fr");
        utilisateur.setMotDePasseHash("hash");
        utilisateur.setRole("USER");

        utilisateurRepository.save(utilisateur);

        Optional<Utilisateur> resultat =
            utilisateurRepository.findByEmail("luffy@test.fr");
        assertTrue(resultat.isPresent());
    }
}

```

Extrait de code 6 : Test d'intégration d'un repository

Cet extrait présente un test d'intégration réalisé sur la couche de persistance. Il vérifie qu'un utilisateur enregistré dans la base de données de test peut être retrouvé grâce à son adresse électronique.

Ce type de test permet de valider le bon fonctionnement des repositories et des relations entre les entités.

Validation des fonctionnalités

Des jeux d'essais ont également été réalisés afin de vérifier le comportement des principales fonctionnalités de l'application dans des conditions proches de son utilisation réelle.

Les vérifications ont notamment porté sur :

- L'inscription des utilisateurs
- L'authentification par JWT
- La création de la progression initiale
- Le déroulement des combats
- L'utilisation des capacités
- La gestion de l'endurance
- Les conditions de victoire ou de défaite
- La progression du joueur

Ces différents tests ont permis d'améliorer la robustesse de l'application et de corriger plusieurs anomalies rencontrées au cours du développement.

7. Conclusion et perspectives

Bilan du projet

La réalisation du projet One Piece Zero m'a permis de mettre en œuvre l'ensemble des compétences visées par le titre Concepteur Développeur d'Applications.

Ce projet m'a conduit à intervenir sur toutes les étapes du développement d'une application : analyse du besoin, rédaction des spécifications, modélisation des données, développement du back-end, réalisation du front-end, sécurisation de l'application, tests et validation des fonctionnalités.

Le développement de l'API REST avec Spring Boot m'a permis d'approfondir mes connaissances en Java, Spring Security, JPA et gestion des bases de données relationnelles. La réalisation du front-end avec Angular m'a également permis de développer des compétences en développement d'interfaces utilisateur et en communication avec une API REST.

Ce projet a également renforcé mes compétences en conception logicielle, en organisation du code et en gestion de projet.

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du développement du projet.

La conception du système de combat a constitué l'une des principales problématiques. La gestion des capacités, des effets temporaires, de l'endurance et des différentes conditions de victoire a nécessité plusieurs évolutions du modèle métier.

La mise en place de l'authentification JWT et de la sécurisation de l'API a également représenté un défi important, notamment lors des premiers tests des routes protégées.

Enfin, l'évolution progressive du modèle de données a conduit à revoir certaines relations entre les entités afin d'adapter le fonctionnement du jeu aux besoins fonctionnels.

Perspectives d'évolution

Bien que la première version du projet permette déjà de jouer et d'effectuer les principales actions prévues, plusieurs évolutions sont envisagées.

Les futures versions pourraient notamment intégrer :

- Un formulaire admin d'ajout de données
- De nouvelles îles et zones

- Un plus grand nombre de personnages jouables
- Des capacités supplémentaires
- Un système d'équipage
- Des combats multi-joueurs
- Un système d'inventaire et d'objets
- Une économie en jeu
- Un déploiement conteneurisé avec Docker

Ces évolutions permettraient d'enrichir l'expérience utilisateur tout en poursuivant le développement des fonctionnalités techniques de l'application.

Ce projet constitue donc une synthèse concrète des compétences acquises durant la formation et l'alternance. Il m'a permis de concevoir, développer, sécuriser et tester une application complète, tout en consolidant ma posture de concepteur développeur d'applications.